

Aplicación de EDAs a la optimización de plantillas SBC en FC24

Josep Gabriel Fornes Reynes

Repositorio del proyecto | Memoria del TFG original

1. Contexto y objetivo

Este trabajo parte de mi Trabajo Final de Grado, centrado en **FC24 Ultimate Team**, donde se abordaba la resolución de desafíos de creación de plantillas (SBC/DCP) mediante técnicas evolutivas [1]. En estos desafíos se debe construir una plantilla de 11 jugadores que cumpla restricciones como media mínima, química, ligas, clubes, nacionalidades, número máximo o mínimo de jugadores por grupo y versiones especiales de carta. Además, la solución debe ser barata, ya que los jugadores entregados desaparecen del club.

El sistema original resolvía el problema con un algoritmo genético (GA): cada individuo era una plantilla, la población se evaluaba mediante una función de *fitness* y se aplicaban selección, cruce, mutación y elitismo. El objetivo de este trabajo es conectar ese problema real con el seminario de Computación Natural sustituyendo o comparando el GA con un algoritmo de estimación de distribuciones (EDA). En particular, se usa UMDA categórico, donde las nuevas soluciones no se generan mediante operadores genéticos, sino aprendiendo una distribución probabilística a partir de las mejores plantillas y muestreando desde ella [3, 4, 5].

Cada solución se representa como $x = (x_1, \dots, x_{11})$, donde x_i es el jugador asignado al slot i . La función de *fitness* replica la lógica del TFG:

$$f(x) = d(x) + c(x) + \lambda u(x),$$

donde $d(x)$ mide la distancia a los requisitos, $c(x)$ el coste normalizado y $u(x)$ el número de requisitos incumplidos. Se conserva el dataset completo, incluyendo cartas distintas de un mismo jugador, pero se repara cada plantilla para no repetir el mismo nombre dentro del once, siguiendo la restricción efectiva del GA original.

2. Modos de evaluación y protocolo experimental

Se mantienen los dos modos de evaluación del TFG: **club** y **BD**. La diferencia no está en los jugadores que puede elegir el algoritmo, sino en la normalización del precio dentro de la *fitness*. En modo **club**, el coste se normaliza con una referencia calculada a partir de los jugadores disponibles en `jugadores.json`, por lo que el precio pesa más. En modo **BD**, se usa una referencia global mucho mayor, precomputada en `config/price_bounds.json`; por tanto, el mismo coste queda relativamente menos penalizado. Así, BD no significa que el algoritmo pueda escoger cualquier jugador de la base de datos, sino que usa la base de datos para escalar el coste.

También se mantiene la lógica original de posiciones: un jugador puede ocupar un slot que no sea su posición natural, pero entonces su química es cero. Esto evita imponer un filtro duro que podría descartar soluciones válidas en retos sin química alta. No obstante, cuando el desafío exige química, la compatibilidad posicional y las combinaciones de liga, club o nacionalidad se vuelven decisivas.

El protocolo usa los mismos 20 desafíos del TFG y cinco semillas por configuración, dando 100 ejecuciones por variante. La fiabilidad se calcula como requisitos cumplidos sobre 380 requisitos evaluados. Además, se registra el coste medio de la mejor plantilla y el tiempo medio de ejecución. El GA se usa como **baseline histórico**; por ello, el tiempo solo se compara entre variantes UMDA ejecutadas en el nuevo repositorio [2].

3. Fase 1: sustitución directa por UMDA

La primera fase evalúa una sustitución directa del GA por UMDA. Cada uno de los 11 slots puede elegir cualquier jugador del dataset. Esta formulación es comparable con la del GA, pero es desfavorable para UMDA: cada variable categórica tiene 1008 valores posibles y el modelo aprende una distribución univariante,

$$P(x) \approx \prod_{i=1}^{11} P_i(x_i).$$

El problema SBC, en cambio, no es univariante: la química depende de combinaciones entre jugadores, la media se calcula sobre la plantilla completa y algunas restricciones exigen cantidades exactas de ligas, nacionalidades o versiones de carta.

La Tabla 1 muestra que UMDA directo no supera al GA. En **club** pierde fiabilidad y además obtiene mayor coste medio. En **BD** reduce ligeramente el coste medio, pero cae de 89,21 % a 82,37 % de fiabilidad. El resultado indica

Modo	Algoritmo	Cumplidos	Fiabilidad	Coste medio	Runtime
club	GA histórico	322/380	84,74 %	23.788,00	–
club	UMDA directo	308/380	81,05 %	29.517,00	22,91s
BD	GA histórico	339/380	89,21 %	77.501,50	–
BD	UMDA directo	313/380	82,37 %	76.819,50	24,39s

Tabla 1: Comparación estricta entre el GA histórico y UMDA directo.

que el problema no era únicamente económico: UMDA directo no encuentra bien combinaciones factibles. El GA tolera mejor el espacio amplio porque opera sobre plantillas completas y puede conservar bloques útiles mediante cruce y elitismo; UMDA, al remuestrear cada slot de forma independiente, tiende a romper esas estructuras.

4. Fase 2: UMDA estructurado

A partir de ese diagnóstico se implementa `umda_structured`. La evaluación, los retos y los datos no cambian; cambia la representación que recibe UMDA. En vez de permitir que cada slot elija entre todos los jugadores, se construye un dominio local de 150 candidatos por slot. Estos dominios priorizan jugadores compatibles con la posición, alineados con los requisitos del reto, cercanos a la media mínima y razonables en precio. También se reserva una parte para jugadores fuera de posición, por lo que la posición no se convierte en una restricción dura.

Esta segunda fase no invalida la primera: ambas responden a preguntas distintas. La fase directa evalúa el cambio de operador evolutivo manteniendo una codificación casi equivalente a la del GA; la fase estructurada evalúa una versión más propia de un EDA, donde se introduce conocimiento del problema antes de estimar la distribución. En EDAs, la representación es parte del método: si el dominio categórico es demasiado disperso, las probabilidades estimadas son poco informativas; si el dominio está estructurado, el aprendizaje probabilístico recibe una señal mucho más útil.

Modo	Algoritmo	Cumplidos	Fiabilidad	Coste medio	Runtime
club	GA histórico	322/380	84,74 %	23.788,00	–
club	UMDA directo	308/380	81,05 %	29.517,00	22,91s
club	UMDA estructurado	342/380	90,00 %	25.956,50	1,78s
BD	GA histórico	339/380	89,21 %	77.501,50	–
BD	UMDA directo	313/380	82,37 %	76.819,50	24,39s
BD	UMDA estructurado	348/380	91,58 %	66.450,00	1,76s

Tabla 2: Resultados finales de GA, UMDA directo y UMDA estructurado.

La Tabla 2 resume el resultado principal. En modo **club**, UMDA estructurado alcanza 342/380 requisitos cumplidos, frente a 322/380 del GA. Su coste medio es superior al del GA, pero inferior al de UMDA directo. En modo **BD**, obtiene 348/380 requisitos cumplidos, 91,58 % de fiabilidad y un coste medio menor que el GA histórico. Además, el runtime medio cae de unos 23–24 segundos a menos de 2 segundos respecto a UMDA directo. Esta comparación temporal solo se realiza entre variantes UMDA, ya que no se dispone de un runtime homogéneo del GA histórico.

4.1. Diagnóstico sin penalización de precio

Para separar el efecto del coste de la dificultad propia de los requisitos, se ejecutó UMDA estructurado sin penalización de precio en la *fitness*. El resultado fue 350/380 requisitos cumplidos, con 92,11 % de fiabilidad y un coste medio de 113.038,50 monedas. Se resolvieron completamente 16 de los 20 retos; los cuatro restantes quedaron a un requisito pendiente: media mínima en *Icon Flash*, *Liga y Nación Mixta* y *TriNación Elite*, y presencia insuficiente de cartas *Dynamic Duos* en *Leyendas Supremas*. Esto indica que parte de los errores proviene del compromiso coste-requisitos, aunque también existen restricciones combinatorias difíciles con el dataset disponible. En consecuencia, este experimento debe interpretarse como una ejecución sin penalización de precio en la *fitness*; la construcción de dominios puede seguir usando el precio como criterio de ordenación.

5. Discusión, limitaciones y trabajo futuro

La lectura metodológica de los experimentos es clara. UMDA directo no supera al GA porque el modelo univariante asume independencia entre slots, mientras que las soluciones buenas dependen de combinaciones de jugadores: química, ligas, nacionalidades, versiones especiales, compatibilidad posicional y media global. En este contexto, elegir buenos jugadores de forma individual no garantiza construir una plantilla válida. El GA histórico tolera mejor esta representación amplia porque evoluciona plantillas completas y puede conservar bloques útiles mediante cruce y elitismo. UMDA, en cambio, remuestrea cada slot de forma independiente y puede romper esas combinaciones.

UMDA estructurado mejora porque reduce el espacio de búsqueda y aumenta la señal estadística disponible para el modelo. Al construir dominios locales por slot, el algoritmo ya no debe descubrir desde cero qué jugadores son razonables para cada slot o reto. Esta mejora no implica que “UMDA gane al GA” en general, sino que un EDA puede ser competitivo cuando el problema se codifica de forma coherente con el modelo probabilístico que aprende. La fase directa y la fase estructurada responden, por tanto, a preguntas distintas: la primera evalúa la sustitución directa del operador evolutivo; la segunda evalúa una formulación más adecuada para un EDA.

Aspecto observado	Interpretación
UMDA directo pierde frente al GA	El dominio completo es demasiado amplio y poco informativo para un modelo univariante.
UMDA estructurado mejora	Los dominios locales reducen evaluaciones poco prometedoras e introducen conocimiento del problema.
BD cambia el equilibrio	El coste se normaliza con una referencia mayor y penaliza menos las plantillas caras.
Persisten fallos residuales	Algunas medias altas o cartas especiales requieren combinaciones muy concretas.

Tabla 3: Interpretación metodológica de los resultados.

También conviene distinguir entre mejorar la fiabilidad y mejorar el coste. En modo **club**, UMDA estructurado mejora claramente el cumplimiento de requisitos, aunque no consigue ser más barato que el GA histórico. En modo **BD**, en cambio, mejora tanto la fiabilidad como el coste medio. Esto se explica porque en BD el coste tiene menos peso relativo dentro de la *fitness*, lo que permite priorizar combinaciones válidas sin penalizar tanto el uso de cartas algo más caras. El experimento sin penalización de precio en la *fitness* refuerza esta interpretación: UMDA estructurado alcanza 350/380 requisitos cumplidos, con una fiabilidad del 92,11 %, aunque el coste medio aumenta hasta 113.038,50 monedas. Por tanto, parte de los errores venían del compromiso coste-requisitos, pero siguen existiendo restricciones combinatorias difíciles con el dataset disponible.

La comparación se realiza tomando el GA del TFG como baseline empírico, ya que fue evaluado sobre los mismos desafíos, métricas y modos de precio. Por tanto, los resultados permiten comparar de forma directa el comportamiento del nuevo enfoque basado en UMDA frente al sistema original. La única precaución es que los tiempos de ejecución no son completamente comparables, porque el GA pertenece al desarrollo previo y no se ha reejecutado en el mismo entorno que las variantes UMDA. Como trabajo futuro, sería interesante reimplementar el GA en el nuevo repositorio y probarlo también con dominios estructurados, pero esto no invalida la comparación principal: UMDA directo no basta, mientras que UMDA estructurado mejora el baseline histórico en fiabilidad y, en modo BD, también en coste medio.

Desde el punto de vista de reproducibilidad, el repositorio incluye el código, los datos de jugadores y retos, los límites de normalización de precio, los CSV crudos de cada ejecución en **results/raw** y los resúmenes agregados en **results/summary**. Esto permite comprobar de dónde salen las tablas y repetir los experimentos con las mismas semillas, lo cual es importante para que la comparación no dependa únicamente de resultados agregados [2].

Como trabajo futuro, convendría: (i) reejecutar el GA en el mismo entorno; (ii) aplicar dominios estructurados también al GA; (iii) probar EDAs multivariantes, por ejemplo basados en redes bayesianas, capaces de capturar dependencias entre slots; (iv) incorporar reparación inteligente o búsqueda local tras el muestreo para corregir media, química y cartas especiales; y (v) estudiar la sensibilidad a **domain_size**, **size_gen** y **max_iter**. Estos experimentos permitirían distinguir con más precisión si la mejora proviene del aprendizaje probabilístico, del preprocesamiento del dominio o de la combinación de ambos factores.

6. Conclusión

El trabajo demuestra que aplicar EDAs al problema de optimización de plantillas SBC no solo es viable, sino que puede mejorar los resultados del algoritmo genético original cuando la representación se adapta al modelo probabilístico utilizado. La sustitución directa del GA por UMDA no mejora el baseline, ya que el modelo univariante no captura las dependencias entre jugadores. Sin embargo, al introducir dominios estructurados por slot y requisitos del desafío, UMDA supera al GA histórico en fiabilidad: pasa de 84,74 % a 90,00 % en modo **club** y de 89,21 % a 91,58 % en modo **BD**. Además, en modo **BD** también reduce el coste medio de la plantilla, de 77.501,50 a 66.450,00 monedas.

Por tanto, el resultado principal del trabajo es que el EDA estructurado mejora el sistema original en los experimentos realizados. La comparación también muestra que esta mejora no se obtiene simplemente sustituyendo cruce y mutación por UMDA, sino formulando el espacio de búsqueda de manera coherente con el aprendizaje probabilístico. En consecuencia, los EDAs no son automáticamente superiores a los GA, pero pueden ser muy competitivos e incluso superar al baseline cuando la representación del problema incorpora conocimiento útil sobre los requisitos del dominio.

Referencias

- [1] J. G. Fornes Reynes. *Tècniques evolutives per a la presa de decisions en videojocs*. Trabajo de Fin de Grado, Universitat de les Illes Balears, 2024–2025. Memoria disponible en: memoria del TFG original.
- [2] J. G. Fornes Reynes. *EDA FC24 Squad Optimizer*. Repositorio de código y resultados experimentales. Disponible en: repositorio GitHub del proyecto.
- [3] H. Mühlenbein and G. Paaß. From recombination of genes to the estimation of distributions I. Binary parameters. In *Parallel Problem Solving from Nature – PPSN IV*, pp. 178–187, 1996.
- [4] P. Larrañaga, H. Karshenas, C. Bielza and R. Santana. A review on probabilistic graphical models in evolutionary computation. *Journal of Heuristics*, 18:795–819, 2012.
- [5] P. Larrañaga. *Estimation of Distribution Algorithms*. Material del Seminario de Computación Natural, Máster Universitario en Inteligencia Artificial.